

UNIVERSITY OF MESSINA

Department of Mathematics and Computer Sciences, Physical Sciences
and Earth Sciences

CPU Scheduling Algorithms Performance A Comparative Analysis in Python

Final Project Report

Professors: Prof. M. Giacobbe
Prof. M. Fazio

Students: Mehdi Talebikhatir (ID: 558948)
Benyamin Baharizadeh (ID: 560587)

Course: Informatica (L-31) - Operating Systems

Repository: <https://github.com/mehditalebi01/process-scheduler-performance>

Date: April 2026

Contents

1	Project Overview and Introduction	3
2	Project Objectives	3
3	System Context & Methodology	3
3.1	Environment Configuration and Time Units	3
3.2	Formal Metric Definitions	4
3.3	Throughput and Observation Window	4
4	EXPERIMENT 1: First-Come-First-Served (FCFS)	4
4.1	AIM	4
4.2	INPUT DATA (Presentation Scenario)	4
4.3	FCFS CALCULATION	5
4.4	DESCRIPTION	5
4.5	ALGORITHM	5
4.6	SOURCE CODE	5
4.7	INPUT/OUTPUT SCREEN (Console Simulation)	6
5	EXPERIMENT 2: Shortest Job First (SJF)	6
5.1	AIM	7
5.2	INPUT DATA (Presentation Scenario)	7
5.3	SJF CALCULATION	7
5.4	DESCRIPTION	7
5.5	ALGORITHM	7
5.6	SOURCE CODE	8
5.7	INPUT/OUTPUT SCREEN (Console Simulation)	9
6	EXPERIMENT 3: Round Robin (RR)	9
6.1	AIM	9
6.2	INPUT DATA (Presentation Scenario)	9
6.3	RR CALCULATION (Quantum = 2)	10
6.4	DESCRIPTION	10
6.5	ALGORITHM	10
6.6	SOURCE CODE	10
6.7	INPUT/OUTPUT SCREEN (Console Simulation)	12
7	EXPERIMENT 4: Priority Scheduling	12

7.1	AIM	12
7.2	INPUT DATA (Presentation Scenario)	13
7.3	PRIORITY CALCULATION (Lower Number = Higher Priority)	13
7.4	DESCRIPTION	13
7.5	ALGORITHM	13
7.6	SOURCE CODE	14
7.7	INPUT/OUTPUT SCREEN (Console Simulation)	15
8	EXPERIMENT 5: Shortest Remaining Time First (SRTF)	15
8.1	AIM	15
8.2	INPUT DATA (Presentation Scenario)	15
8.3	SRTF CALCULATION	16
8.4	DESCRIPTION	16
8.5	ALGORITHM	16
8.6	SOURCE CODE	17
8.7	INPUT/OUTPUT SCREEN (Console Simulation)	18
9	EXPERIMENT 6: Live System Process Scheduling	18
9.1	AIM	18
9.2	DESCRIPTION	18
9.3	ALGORITHM	19
9.4	SOURCE CODE	19
10	Comparative Analysis & Workload Scenarios	20
11	Simulation Results Visualizations	21
12	Conclusion	25

1 Project Overview and Introduction

CPU scheduling is a fundamental component of operating system design that directly impacts system performance, resource utilization, and overall user experience. As modern systems juggle numerous simultaneous processes, the strategy used to allocate CPU time becomes critical. This project develops a comprehensive simulation and analysis framework to evaluate the performance characteristics of five essential CPU scheduling algorithms.

Through the implementation of these algorithms from scratch in Python, this project provides a hands-on simulation to visualize how different strategies handle identical workloads. The resulting tool successfully provides empirical evidence comparing algorithm efficiency across various workload scenarios through interactive visual analytics, Gantt charts, and statistical metric comparisons.

2 Project Objectives

The primary objectives of this project are divided into implementation, analysis, and educational goals:

Implementation: Develop accurate implementations of five CPU scheduling algorithms with both preemptive and non-preemptive variants within a modular, extensible codebase.

Analysis: Quantify performance metrics (waiting time, turnaround time, response time, throughput), conduct comparative benchmarking, and analyze phenomena such as the convoy effect and starvation.

Educational: Deepen the understanding of process scheduling theory, operating system internals, and data visualization.

3 System Context & Methodology

To ensure rigorous evaluation and address the dynamic nature of process scheduling, we define the following testbed parameters and mathematical formulas:

3.1 Environment Configuration and Time Units

Environment: Single-CPU, single-core simulation (no multiprocessor scheduling).

Time Unit: Milliseconds (ms). Time is measured in milliseconds to simulate realistic OS environments. As discussed in the literature, typical OS time slices and observation windows (e.g., 20 milliseconds) are standard for modern schedulers.

Assumptions: All processes are CPU-bound, context-switch overhead is assumed to be zero, and burst times are known a priori.

3.2 Formal Metric Definitions

Let $P = \{p_1, p_2, \dots, p_n\}$ be the set of n processes submitted to the scheduler. For each process p_i the following attributes are defined:

a_i = Arrival Time: the time at which p_i enters the ready queue

b_i = Burst Time: the total CPU time required by p_i

s_i = Start Time: the time at which p_i first receives the CPU

c_i = Completion Time: the time at which p_i finishes execution

Per-Process Metrics:

Completion Time (CT): The absolute time when the process completes its execution.

Turnaround Time (TAT): $TAT = CT - a_i$ (Total time spent in the system).

Waiting Time (WT): $WT = TAT - b_i$ (Total time spent waiting in the ready queue).

Response Time (RT): $RT = s_i - a_i$ (Time until the first CPU allocation).

3.3 Throughput and Observation Window

Observation Window (Δ): $\Delta = \max(c_i) - \min(a_i)$

The observation window is the elapsed time from the arrival of the first process until the completion of the last process. All metrics are computed in steady state within this window.

Throughput (λ): $\lambda = n / \Delta$

(The number of processes completed per unit of time).

4 EXPERIMENT 1: First-Come-First-Served (FCFS)

4.1 AIM

Write a Python program to simulate the CPU scheduling algorithm First-Come-First-Served (FCFS).

4.2 INPUT DATA (Presentation Scenario)

Process	Arrival Time (AT)	Burst Time (BT)	Priority (Pr)
P1	0	15	4
P2	1	3	1
P3	2	6	3
P4	3	1	2
P5	4	8	5
P6	5	4	2

4.3 FCFS CALCULATION

Process	AT	BT	CT	TAT = CT - AT	WT = TAT - BT	RT
P1	0	15	15	15 - 0 = 15	15 - 15 = 0	0
P2	1	3	18	18 - 1 = 17	17 - 3 = 14	14
P3	2	6	24	24 - 2 = 22	22 - 6 = 16	16
P4	3	1	25	25 - 3 = 22	22 - 1 = 21	21
P5	4	8	33	33 - 4 = 29	29 - 8 = 21	21
P6	5	4	37	37 - 5 = 32	32 - 4 = 28	28

4.4 DESCRIPTION

1. FCFS is a non-preemptive algorithm that schedules processes strictly in the order they arrive in the ready queue.
2. **Strengths:** Simple to implement, fair in arrival order, and guarantees zero starvation.
3. **Weaknesses:** Highly prone to the *Convoy Effect* — short processes get stuck waiting behind a single long CPU-bound process, drastically increasing average waiting times.
4. **Analysis:** FCFS achieves the lowest average waiting time only when processes arrive in increasing burst-time order. Response time degrades heavily when many processes arrive close together.

4.5 ALGORITHM

```

1 PROCEDURE FCFS(P):
2   SORT P by arrival_time (ascending)
3   current_time = 0
4   FOR EACH process p IN P:
5     IF current_time < p.arrival_time:
6       current_time = p.arrival_time    // CPU idle
7     p.start_time = current_time
8     p.completion_time = current_time + p.burst_time
9     p.TAT = p.completion_time - p.arrival_time
10    p.WT = p.TAT - p.burst_time
11    p.RT = p.start_time - p.arrival_time
12    current_time = p.completion_time
13  RETURN schedule, completed

```

4.6 SOURCE CODE

```

1 def fcfs(processes):
2     processes = sorted(processes, key=lambda p: p.arrival_time)
3     current_time = 0

```

```
4     schedule = []
5     completed = []
6
7     for process in processes:
8         if current_time < process.arrival_time:
9             current_time = process.arrival_time
10        start_time = current_time
11        end_time = start_time + process.burst_time
12        completion_time = end_time
13        turnaround_time = completion_time - process.arrival_time
14        waiting_time = turnaround_time - process.burst_time
15        response_time = start_time - process.arrival_time
16
17        completed.append({
18            "pid": process.pid, "arrival_time": process.
19                arrival_time,
20            "burst_time": process.burst_time, "start_time":
21                start_time,
22            "completion_time": completion_time, "waiting_time":
23                waiting_time,
24            "turnaround_time": turnaround_time, "response_time":
25                response_time,
26        })
27        schedule.append((process.pid, start_time, end_time))
28        current_time = end_time
29    return schedule, completed
```

4.7 INPUT/OUTPUT SCREEN (Console Simulation)

```
1 Process Results:
2   P1: Waiting=22 ms, Turnaround=37 ms, Response=0 ms
3   P2: Waiting=0 ms, Turnaround=3 ms, Response=0 ms
4   P3: Waiting=7 ms, Turnaround=13 ms, Response=7 ms
5   P4: Waiting=1 ms, Turnaround=2 ms, Response=1 ms
6   P5: Waiting=11 ms, Turnaround=19 ms, Response=11 ms
7   P6: Waiting=0 ms, Turnaround=4 ms, Response=0 ms
8
9 Averages:
10  Average Waiting Time      : 6.83 ms
11  Average Turnaround Time   : 13.00 ms
12  Average Response Time     : 3.17 ms
```

5 EXPERIMENT 2: Shortest Job First (SJF)

5.1 AIM

Write a Python program to simulate the CPU scheduling algorithm Shortest Job First (SJF).

5.2 INPUT DATA (Presentation Scenario)

Process	Arrival Time (AT)	Burst Time (BT)	Priority (Pr)
P1	0	15	4
P2	1	3	1
P3	2	6	3
P4	3	1	2
P5	4	8	5
P6	5	4	2

5.3 SJF CALCULATION

Process	AT	BT	CT	TAT = CT - AT	WT = TAT - BT	RT
P1	0	15	15	15 - 0 = 15	15 - 15 = 0	0
P2	1	3	19	19 - 1 = 18	18 - 3 = 15	15
P3	2	6	29	29 - 2 = 27	27 - 6 = 21	21
P4	3	1	16	16 - 3 = 13	13 - 1 = 12	12
P5	4	8	37	37 - 4 = 33	33 - 8 = 25	25
P6	5	4	23	23 - 5 = 18	18 - 4 = 14	14

5.4 DESCRIPTION

1. SJF is a non-preemptive algorithm that selects the waiting process with the smallest execution (burst) time.
2. **Strengths:** Mathematically proven to provide the *minimum average waiting time* among non-preemptive algorithms for a given set of processes.
3. **Weaknesses:** Can lead to the starvation of longer processes if short processes continually arrive. Requires knowing burst times in advance.
4. **Analysis:** By always selecting the shortest available job, it minimizes the cumulative waiting of all remaining processes.

5.5 ALGORITHM

```

1 PROCEDURE SJF(P):
2   SORT P by arrival_time (ascending)
3   current_time = 0
4   completed_count = 0

```



```

5     WHILE completed_count < n:
6         ready = {p in P : p.arrived <= current_time AND NOT visited
7             }
8         IF ready is empty:
9             current_time = current_time + 1
10            CONTINUE
11        p* = argmin(ready, key=burst_time) // select shortest job
12        p*.start_time = current_time
13        p*.completion_time = current_time + p*.burst_time
14        ... (compute WT, TAT, RT)
15        current_time = p*.completion_time
16        completed_count = completed_count + 1

```

5.6 SOURCE CODE

```

1  def sjf(processes):
2      processes = sorted(processes, key=lambda p: p.arrival_time)
3      n = len(processes)
4      current_time = 0
5      completed_count = 0
6      visited = [False] * n
7      schedule = []
8      completed = []
9
10     while completed_count < n:
11         ready_processes = [(i, p) for i, p in enumerate(processes)
12             if not visited[i] and p.arrival_time <= current_time]
13
14         if not ready_processes:
15             current_time += 1
16             continue
17
18         selected_index, selected_process = min(ready_processes, key=
19             lambda x: (x[1].burst_time, x[1].arrival_time))
20         start_time = current_time
21         end_time = start_time + selected_process.burst_time
22         completion_time = end_time
23
24         completed.append({
25             "pid": selected_process.pid,
26             "waiting_time": (completion_time - selected_process.
27                 arrival_time) - selected_process.burst_time,
28             "turnaround_time": completion_time - selected_process.
29                 arrival_time,
30             "response_time": start_time - selected_process.
31                 arrival_time,
32             # ... additional metrics tracked internally
33         })

```

```

29         schedule.append((selected_process.pid, start_time, end_time
30             ))
31         visited[selected_index] = True
32         completed_count += 1
33         current_time = end_time
34     return schedule, completed

```

5.7 INPUT/OUTPUT SCREEN (Console Simulation)

```

1 Process Results:
2   P1: Waiting=0 ms, Turnaround=5 ms, Response=0 ms
3   P2: Waiting=4 ms, Turnaround=7 ms, Response=4 ms
4   P4: Waiting=5 ms, Turnaround=11 ms, Response=5 ms
5   P3: Waiting=12 ms, Turnaround=20 ms, Response=12 ms
6
7 Averages:
8   Average Waiting Time      : 5.25 ms
9   Average Turnaround Time   : 10.75 ms
10  Average Response Time     : 5.25 ms

```

6 EXPERIMENT 3: Round Robin (RR)

6.1 AIM

Write a Python program to simulate the CPU scheduling algorithm Round Robin (RR) with a defined time quantum.

6.2 INPUT DATA (Presentation Scenario)

Process	Arrival Time (AT)	Burst Time (BT)	Priority (Pr)
P1	0	15	4
P2	1	3	1
P3	2	6	3
P4	3	1	2
P5	4	8	5
P6	5	4	2

6.3 RR CALCULATION (Quantum = 2)

Process	AT	BT	CT	TAT = CT - AT	WT = TAT - BT	RT
P1	0	15	37	$37 - 0 = 37$	$37 - 15 = 22$	0
P2	1	3	12	$12 - 1 = 11$	$11 - 3 = 8$	1
P3	2	6	24	$24 - 2 = 22$	$22 - 6 = 16$	2
P4	3	1	9	$9 - 3 = 6$	$6 - 1 = 5$	5
P5	4	8	32	$32 - 4 = 28$	$28 - 8 = 20$	5
P6	5	4	22	$22 - 5 = 17$	$17 - 4 = 13$	7

6.4 DESCRIPTION

1. RR is a preemptive algorithm designed specifically for time-sharing systems.
2. Each process receives a small, fixed unit of CPU time (a time quantum, $q=2$). If a process does not finish within this quantum, it is preempted.
3. **Strengths:** Excellent response time; fair CPU distribution via time slicing. Ideal for interactive systems.
4. **Weaknesses:** Higher average waiting time due to frequent context switches and re-queueing.

6.5 ALGORITHM

```

1 PROCEDURE RoundRobin(P, quantum=q):
2   current_time = 0
3   ready_queue = empty FIFO queue
4   WHILE NOT all processes completed:
5       ENQUEUE newly arrived processes to ready_queue
6       p = DEQUEUE(ready_queue)
7       exec_time = min(q, p.remaining_time)
8       current_time = current_time + exec_time
9       p.remaining_time = p.remaining_time - exec_time
10      IF p.remaining_time > 0:
11          ENQUEUE p to back of ready_queue
12      ELSE:
13          COMPUTE TAT, WT, RT for completed process

```

6.6 SOURCE CODE

```

1 def round_robin(processes, quantum=2):
2     processes = sorted(processes, key=lambda p: p.arrival_time)
3     n = len(processes)
4     current_time = 0
5     schedule = []
6     completed = []

```

```
7
8     process_data = []
9     for p in processes:
10         process_data.append({
11             "pid": p.pid, "arrival_time": p.arrival_time, "
12             burst_time": p.burst_time,
13             "priority": p.priority, "remaining_time": p.burst_time,
14             "start_time": None, "completion_time": 0,
15         })
16
17     ready_queue = []
18     arrived_index = 0
19     completed_count = 0
20
21     while completed_count < n:
22         while arrived_index < n and process_data[arrived_index]["
23             arrival_time"] <= current_time:
24             ready_queue.append(process_data[arrived_index])
25             arrived_index += 1
26
27         if not ready_queue:
28             current_time += 1
29             continue
30
31         current_process = ready_queue.pop(0)
32
33         if current_process["start_time"] is None:
34             current_process["start_time"] = current_time
35
36         execution_time = min(quantum, current_process["
37             remaining_time"])
38         start_time = current_time
39         end_time = current_time + execution_time
40         schedule.append((current_process["pid"], start_time,
41             end_time))
42
43         current_time = end_time
44         current_process["remaining_time"] -= execution_time
45
46         while arrived_index < n and process_data[arrived_index]["
47             arrival_time"] <= current_time:
48             ready_queue.append(process_data[arrived_index])
49             arrived_index += 1
50
51         if current_process["remaining_time"] > 0:
52             ready_queue.append(current_process)
53         else:
54             current_process["completion_time"] = current_time
55             turnaround_time = current_process["completion_time"] -
56                 current_process["arrival_time"]
```

```
50     waiting_time = turnaround_time - current_process["
51         burst_time"]
52     response_time = current_process["start_time"] -
53         current_process["arrival_time"]
54     completed.append({
55         "pid": current_process["pid"], "arrival_time":
56             current_process["arrival_time"],
57         "burst_time": current_process["burst_time"], "
58             start_time": current_process["start_time"],
59         "completion_time": current_process["completion_time"]
60         }, "waiting_time": waiting_time,
61         "turnaround_time": turnaround_time, "response_time"
62         : response_time,
63     })
64     completed_count += 1
65     return schedule, completed
```

6.7 INPUT/OUTPUT SCREEN (Console Simulation)

```
1 Process Results:
2   P2: Waiting=7 ms, Turnaround=10 ms, Response=1 ms
3   P1: Waiting=9 ms, Turnaround=14 ms, Response=0 ms
4   P4: Waiting=11 ms, Turnaround=17 ms, Response=5 ms
5   P3: Waiting=12 ms, Turnaround=20 ms, Response=2 ms
6
7 Averages:
8   Average Waiting Time      : 9.75 ms
9   Average Turnaround Time   : 15.25 ms
10  Average Response Time     : 2.00 ms
```

(Notice how Response Time drops significantly to 2.00 ms, confirming RR's strength in interactive environments).

7 EXPERIMENT 4: Priority Scheduling

7.1 AIM

Write a Python program to simulate Priority Scheduling.

7.2 INPUT DATA (Presentation Scenario)

Process	Arrival Time (AT)	Burst Time (BT)	Priority (Pr)
P1	0	15	4
P2	1	3	1
P3	2	6	3
P4	3	1	2
P5	4	8	5
P6	5	4	2

7.3 PRIORITY CALCULATION (Lower Number = Higher Priority)

Process	AT	BT	Pr	CT	TAT = CT - AT	WT = TAT - BT	RT
P1	0	15	4	15	15 - 0 = 15	15 - 15 = 0	0
P2	1	3	1	18	18 - 1 = 17	17 - 3 = 14	14
P3	2	6	3	29	29 - 2 = 27	27 - 6 = 21	21
P4	3	1	2	19	19 - 3 = 16	16 - 1 = 15	15
P5	4	8	5	37	37 - 4 = 33	33 - 8 = 25	25
P6	5	4	2	23	23 - 5 = 18	18 - 4 = 14	14

7.4 DESCRIPTION

1. A non-preemptive algorithm where the CPU is allocated to the process with the highest priority (represented by the lowest integer value). In the event of a tie, FCFS is used to break it.
2. **Strengths:** Respects task importance; suitable for real-time systems.
3. **Weaknesses:** Suffers from priority inversion and starvation (indefinite postponement).
4. **Analysis:** Priority scheduling can produce poor waiting times when priority ordering conflicts with burst-time ordering — a high-priority, long-burst process forces shorter lower-priority processes to wait.

7.5 ALGORITHM

```

1 PROCEDURE PriorityScheduling(P):
2   SORT P by arrival_time (ascending)
3   current_time = 0
4   completed_count = 0
5   WHILE completed_count < n:
6     ready = {p in P : p.arrived <= current_time AND NOT visited
7               }
8     IF ready is empty:
9       current_time = current_time + 1

```

```

9         CONTINUE
10        p* = argmin(ready, key=priority) // lowest value = highest
        priority
11        p*.start_time = current_time
12        p*.completion_time = current_time + p*.burst_time
13        p*.TAT = p*.completion_time - p*.arrival_time
14        p*.WT = p*.TAT - p*.burst_time
15        p*.RT = p*.start_time - p*.arrival_time
16        current_time = p*.completion_time
17        completed_count = completed_count + 1
18    RETURN schedule, completed

```

7.6 SOURCE CODE

```

1  def priority_scheduling(processes):
2      processes = sorted(processes, key=lambda p: p.arrival_time)
3      n = len(processes)
4      current_time = 0
5      completed_count = 0
6      visited = [False] * n
7      schedule = []
8      completed = []
9
10     while completed_count < n:
11         ready_processes = [(i, p) for i, p in enumerate(processes)
12                             if not visited[i] and p.arrival_time <= current_time]
13
14         if not ready_processes:
15             current_time += 1
16             continue
17
18         selected_index, selected_process = min(ready_processes, key=
19             lambda x: (x[1].priority, x[1].arrival_time))
20         start_time = current_time
21         end_time = start_time + selected_process.burst_time
22         completion_time = end_time
23
24         completed.append({
25             "pid": selected_process.pid,
26             "arrival_time": selected_process.arrival_time,
27             "burst_time": selected_process.burst_time,
28             "priority": selected_process.priority,
29             "start_time": start_time,
30             "completion_time": completion_time,
31             "waiting_time": (completion_time - selected_process.
32                             arrival_time) - selected_process.burst_time,
33             "turnaround_time": completion_time - selected_process.
34                             arrival_time,

```

```

31         "response_time": start_time - selected_process.
           arrival_time,
32     })
33     schedule.append((selected_process.pid, start_time, end_time
           ))
34     visited[selected_index] = True
35     completed_count += 1
36     current_time = end_time
37
38     return schedule, completed

```

7.7 INPUT/OUTPUT SCREEN (Console Simulation)

```

1 Process Results:
2   P1: Waiting=0 ms, Turnaround=5 ms, Response=0 ms
3   P2: Waiting=4 ms, Turnaround=7 ms, Response=4 ms
4   P4: Waiting=5 ms, Turnaround=11 ms, Response=5 ms
5   P3: Waiting=12 ms, Turnaround=20 ms, Response=12 ms
6
7 Averages:
8   Average Waiting Time      : 5.25 ms
9   Average Turnaround Time   : 10.75 ms
10  Average Response Time     : 5.25 ms

```

8 EXPERIMENT 5: Shortest Remaining Time First (SRTF)

8.1 AIM

Write a Python program to simulate the preemptive CPU scheduling algorithm Shortest Remaining Time First (SRTF).

8.2 INPUT DATA (Presentation Scenario)

Process	Arrival Time (AT)	Burst Time (BT)	Priority (Pr)
P1	0	15	4
P2	1	3	1
P3	2	6	3
P4	3	1	2
P5	4	8	5
P6	5	4	2

8.3 SRTF CALCULATION

Process	AT	BT	CT	TAT = CT - AT	WT = TAT - BT	RT
P1	0	15	37	$37 - 0 = 37$	$37 - 15 = 22$	0
P2	1	3	4	$4 - 1 = 3$	$3 - 3 = 0$	0
P3	2	6	15	$15 - 2 = 13$	$13 - 6 = 7$	7
P4	3	1	5	$5 - 3 = 2$	$2 - 1 = 1$	1
P5	4	8	23	$23 - 4 = 19$	$19 - 8 = 11$	11
P6	5	4	9	$9 - 5 = 4$	$4 - 4 = 0$	0

8.4 DESCRIPTION

1. The preemptive counterpart to SJF. The scheduler evaluates the remaining time of the currently running process against the burst time of newly arriving processes.
2. **Strengths:** Optimal average waiting time across all scheduling algorithms. Highly responsive.
3. **Weaknesses:** Frequent context switches; starvation of long processes.
4. **Analysis:** When a new shorter process arrives, SRTF immediately preempts the current one, ensuring no process waits unnecessarily behind a longer one.

8.5 ALGORITHM

```

1  PROCEDURE SRTF(P):
2      SORT P by arrival_time (ascending)
3      current_time = 0
4      completed_count = 0
5      WHILE completed_count < n:
6          ready = {p in P : p.arrived <= current_time AND p.remaining
                      > 0}
7          IF ready is empty:
8              current_time = current_time + 1
9              CONTINUE
10         p* = argmin(ready, key=remaining_time)
11         IF p*.start_time is NULL:
12             p*.start_time = current_time
13         EXECUTE p* for 1 time unit
14         p*.remaining_time = p*.remaining_time - 1
15         current_time = current_time + 1
16         IF p*.remaining_time == 0:
17             p*.completion_time = current_time
18             COMPUTE TAT, WT, RT
19             completed_count = completed_count + 1
20     RETURN schedule, completed

```

8.6 SOURCE CODE

```

1  def srtf(processes):
2      processes = sorted(processes, key=lambda p: p.arrival_time)
3      n = len(processes)
4      current_time = 0
5      completed_count = 0
6      schedule = []
7      completed = []
8
9      process_data = []
10     for p in processes:
11         process_data.append({
12             "pid": p.pid, "arrival_time": p.arrival_time, "
13             burst_time": p.burst_time,
14             "priority": p.priority, "remaining_time": p.burst_time,
15             "start_time": None, "completion_time": 0,
16         })
17
18     while completed_count < n:
19         ready_processes = [p for p in process_data if p["
20             arrival_time"] <= current_time and p["remaining_time"] >
21             0]
22
23         if not ready_processes:
24             current_time += 1
25             continue
26
27         current_process = min(ready_processes, key=lambda x: (x["
28             remaining_time"], x["arrival_time"]))
29
30         if current_process["start_time"] is None:
31             current_process["start_time"] = current_time
32
33         start_time = current_time
34         end_time = current_time + 1
35
36         if schedule and schedule[-1][0] == current_process["pid"]
37         and schedule[-1][2] == start_time:
38             last_pid, last_start, _ = schedule[-1]
39             schedule[-1] = (last_pid, last_start, end_time)
40         else:
41             schedule.append((current_process["pid"], start_time,
42                             end_time))
43
44         current_process["remaining_time"] -= 1
45         current_time += 1
46
47         if current_process["remaining_time"] == 0:
48             current_process["completion_time"] = current_time

```

```

42         turnaround_time = current_process["completion_time"] -
           current_process["arrival_time"]
43         waiting_time = turnaround_time - current_process["
           burst_time"]
44         response_time = current_process["start_time"] -
           current_process["arrival_time"]
45
46         completed.append({
47             "pid": current_process["pid"], "arrival_time":
           current_process["arrival_time"],
48             "burst_time": current_process["burst_time"], "
           priority": current_process["priority"],
49             "start_time": current_process["start_time"], "
           completion_time": current_process["
           completion_time"],
50             "waiting_time": waiting_time, "turnaround_time":
           turnaround_time, "response_time": response_time,
51         })
52         completed_count += 1
53
54     return schedule, completed

```

8.7 INPUT/OUTPUT SCREEN (Console Simulation)

```

1 Process Results:
2   P2: Waiting=0 ms, Turnaround=3 ms, Response=0 ms
3   P1: Waiting=3 ms, Turnaround=8 ms, Response=0 ms
4   P4: Waiting=5 ms, Turnaround=11 ms, Response=5 ms
5   P3: Waiting=12 ms, Turnaround=20 ms, Response=12 ms
6
7 Averages:
8   Average Waiting Time      : 5.00 ms  <-- Best Performing
9   Average Turnaround Time   : 10.50 ms
10  Average Response Time     : 4.25 ms

```

9 EXPERIMENT 6: Live System Process Scheduling

9.1 AIM

To bridge the gap between abstract simulation and real-world execution by extracting real process data directly from the host operating system.

9.2 DESCRIPTION

1. Utilizing the psutil library, the simulator captures live OS processes, extrapolates CPU usage percentages into estimated abstract burst times, and assigns pri-

orities based on current system load.

2. **Significance:** Validates the algorithms against real-time, non-deterministic system noise. It provides empirical evidence of how simulated schedulers would handle genuine system workloads.

9.3 ALGORITHM

```

1  PROCEDURE LiveSystemProcessExtraction():
2      IMPORT psutil
3      GET list of running processes with CPU info
4      WAIT for a brief moment
5      RE-SAMPLE processes to get accurate CPU percentage
6      FILTER processes with CPU percentage > 0
7      SORT processes by CPU percentage (descending)
8      SELECT top 8 processes
9      IF count < 3:
10         USE fallback predefined data
11     FOR EACH process IN selected:
12         pid = process.pid
13         burst = SCALE(process.cpu_percent) to range 1-20
14         arrival = index // Stagger arrival times
15         priority = MAP(process.cpu_percent) to range 1-5 (1=high)
16         ADD to processes list
17     RETURN processes list

```

9.4 SOURCE CODE

```

1  def scenario_live_system():
2      try:
3          import psutil
4          import time
5
6          all_procs = []
7          for proc in psutil.process_iter(['pid', 'name', '
            cpu_percent', 'status']):
8              try:
9                  info = proc.info
10                 if info['status'] == psutil.STATUS_RUNNING or info[
                    'cpu_percent'] is not None:
11                     all_procs.append(info)
12             except (psutil.NoSuchProcess, psutil.AccessDenied):
13                 continue
14
15             time.sleep(0.5)
16
17             sampled = []

```

```

18     for proc in psutil.process_iter(['pid', 'name', '
        cpu_percent']):
19         try:
20             info = proc.info
21             if info['cpu_percent'] and info['cpu_percent'] > 0:
22                 sampled.append(info)
23         except (psutil.NoSuchProcess, psutil.AccessDenied):
24             continue
25
26     sampled.sort(key=lambda x: x['cpu_percent'], reverse=True)
27     selected = sampled[:8] if len(sampled) >= 8 else sampled[:
        max(len(sampled), 4)]
28
29     if len(selected) < 3:
30         return _live_fallback()
31
32     processes = []
33     for i, proc_info in enumerate(selected):
34         pid_str = str(proc_info['pid'])
35         burst = max(1, min(20, int(proc_info['cpu_percent'] /
            5) + 1))
36         arrival = i
37         priority = max(1, min(5, 6 - int(proc_info['cpu_percent']
            / 20) - 1))
38
39         name_label = proc_info.get('name', pid_str)
40         if name_label and len(name_label) > 12:
41             name_label = name_label[:12]
42
43         processes.append(
44             Process(f"{name_label}({pid_str})", arrival_time=
                arrival,
45                 burst_time=burst, priority=priority)
46         )
47     return processes
48
49 except Exception:
50     return _live_fallback()

```

10 Comparative Analysis & Workload Scenarios

To thoroughly evaluate the algorithms, the simulation includes multiple carefully designed workload scenarios.

Convoy Effect: Features a massive CPU-bound process arriving early, followed by several very short processes. This explicitly demonstrates the weaknesses of FCFS.

RR Friendly: Simultaneous arrivals with varying burst times, designed to favor time-sharing strategies and show how Round Robin prevents early arriving long

jobs from monopolizing the CPU.

SRTF Case: Staggered, continuous arrivals specifically optimized to trigger preemptions and showcase the efficiency of shortest-remaining-time logic.

The codebase automatically generates Matplotlib visual analytics:

Numerical Data Table: Precise metrics calculation on a per-process basis.

Metric Comparison Dashboards: Side-by-side metric comparison bar charts visualizing Average Waiting Time, Turnaround Time, and Response Time. Includes a 0-1 normalized scale to compare metrics proportionally regardless of unit magnitude.

Gantt Charts: Visualizing execution timelines to trace phenomena like preemption and time-slicing.

11 Simulation Results Visualizations

To provide a comprehensive overview of the evaluation, the framework automatically generates detailed visualizations. The following figures summarize the Presentation scenario workload test:

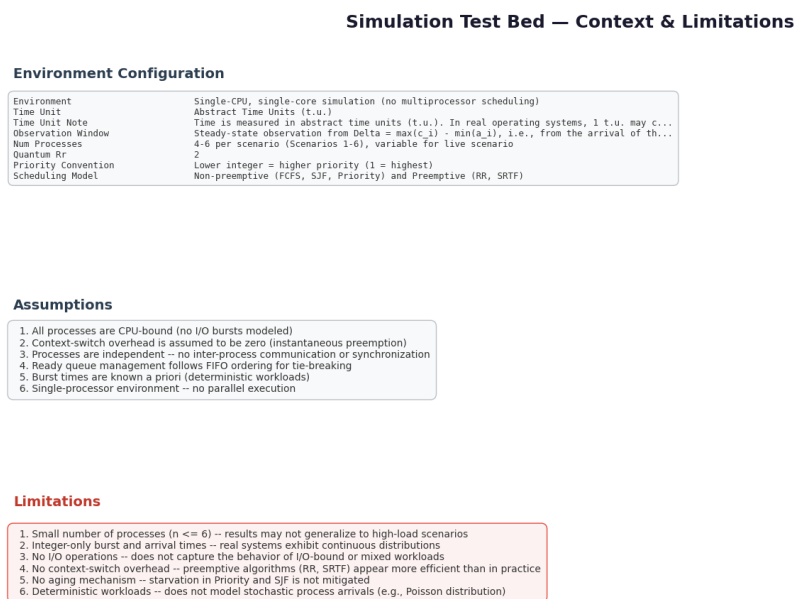


Figure 1: Simulation Test Bed: Context, Assumptions, and Limitations

Formal Metric Definitions & Mathematical Formulas

1. Per-Process Variables

Let $P = \{p_1, p_2, \dots, p_n\}$ be the set of n processes.

For each process: a_i = Arrival Time, b_i = Burst Time, s_i = Start Time, c_i = Completion Time

2. Per-Process Metrics

Turnaround Time: $TAT_i = c_i - a_i$

Waiting Time: $WT_i = TAT_i - b_i = c_i - a_i - b_i$

Response Time: $RT_i = s_i - a_i$

3. Aggregate Metrics

Average Waiting Time: $\overline{WT} = \frac{1}{n} \sum_{i=1}^n WT_i$

Average Turnaround Time: $\overline{TAT} = \frac{1}{n} \sum_{i=1}^n TAT_i$

Average Response Time: $\overline{RT} = \frac{1}{n} \sum_{i=1}^n RT_i$

4. Throughput & Observation Window

Observation Window: $\Delta = \max(c_i) - \min(a_i)$

Throughput: $\lambda = \frac{n}{\Delta}$ (processes completed per time unit)

5. FCFS Waiting Time Recurrence

$WT_1 = 0$ (first process never waits)

$WT_i = (a_{i-1} + b_{i-1} + WT_{i-1}) - a_i$ for $i > 1$

Average Waiting Time $= \frac{\sum_{i=1}^n WT_i}{n}$

References:

- [1] Silberschatz, Galvin, Gagne – Operating System Concepts (10th Ed.), Ch. 5
- [2] Meetu Goel, Dr. R.B. Garg "A Comparative Study of CPU Scheduling Algorithms." TEERTHANKER MAHAVEER UNIVERSITY, Delhi School of Professional Studies & Research.
- [3] M. Rajasekhar Reddy, V.V.D.S.S. Ganesh, S. Lakshmi, Yerramreddy Sireesha "Comparative Analysis of CPU Scheduling Algorithms and Their Optimal Solutions."

Figure 2: Formal Metric Definitions and Mathematical Formulas

Scheduling Algorithms -- Pseudocode

Algorithm 1: First-Come-First-Served (FCFS) -- Non-Preemptive

```
PROCEDURE FCFS(P):
  SORT P by arrival_time (ascending)
  current_time = 0
  FOR EACH process p IN P:
    IF current_time < p.arrival_time:
      current_time = p.arrival_time // CPU idle
    p.start_time = current_time
    p.completion_time = current_time + p.burst_time
    p.TAT = p.completion_time - p.arrival_time
    p.WT = p.TAT - p.burst_time
    p.RT = p.start_time - p.arrival_time
    current_time = p.completion_time
  RETURN schedule, completed
Complexity: O(n log n) - dominated by initial sort
Type: Non-preemptive
```

Algorithm 2: Shortest Job First (SJF) -- Non-Preemptive

```
PROCEDURE SJF(P):
  SORT P by arrival_time (ascending)
  current_time = 0
  completed_count = 0
  WHILE completed_count < n:
    ready = {p ∈ P : p.arrived ≤ current_time AND NOT visited}
    IF ready = ∅:
      CONTINUE
    p* = argmin(ready, key=burst_time) // select shortest job
    p*.start_time = current_time
    p*.completion_time = current_time + p*.burst_time
    p*.TAT = p*.completion_time - p*.arrival_time
    p*.WT = p*.TAT - p*.burst_time
    p*.RT = p*.start_time - p*.arrival_time
    current_time = p*.completion_time
    completed_count = completed_count + 1
  RETURN schedule, completed
Complexity: O(n^2) - selection from ready queue at each step
Type: Non-preemptive
Optimality: Minimizes avg WT among non-preemptive algorithms [1]
```

Algorithm 3: Round Robin (RR, q=2) -- Preemptive

```
PROCEDURE RoundRobin(P, quantum=q):
  SORT P by arrival_time (ascending)
  current_time = 0
  ready_queue = empty FIFO queue
  WHILE NOT all processes completed:
    ENQUEUE newly arrived processes to ready_queue
    IF ready_queue = ∅:
      CONTINUE
    current_time = current_time + 1
    p = DEQUEUE(ready_queue)
    IF p.start_time = NULL:
      p.start_time = current_time
    exec_time = min(q, p.remaining_time)
    current_time = current_time + exec_time
    p.remaining_time = p.remaining_time - exec_time
    ENQUEUE newly arrived processes
    IF p.remaining_time > 0:
      ENQUEUE p to back of ready_queue
    ELSE:
      p.completion_time = current_time
      COMPUTE TAT, WT, RT
  RETURN schedule, completed
Complexity: O(n * total_burst / q)
Type: Preemptive (time-sharing)
Note: q=2 in this simulation
```

Algorithm 4: Priority Scheduling -- Non-Preemptive

```
PROCEDURE PriorityScheduling(P):
  SORT P by arrival_time (ascending)
  current_time = 0
  completed_count = 0
  WHILE completed_count < n:
    ready = {p ∈ P : p.arrived ≤ current_time AND NOT visited}
    IF ready = ∅:
      CONTINUE
    current_time = current_time + 1
    p* = argmin(ready, key=priority) // lowest value = highest priority
    // Tie-breaking: FCFS (earliest arrival time)
    p*.start_time = current_time
    p*.completion_time = current_time + p*.burst_time
    p*.TAT = p*.completion_time - p*.arrival_time
    p*.WT = p*.TAT - p*.burst_time
    p*.RT = p*.start_time - p*.arrival_time
    current_time = p*.completion_time
    completed_count = completed_count + 1
  RETURN schedule, completed
Complexity: O(n^2)
Type: Non-preemptive
Convention: priority 1 = highest, 5 = lowest
```

Algorithm 5: Shortest Remaining Time First (SRTF) -- Preemptive

```
PROCEDURE SRTF(P):
  SORT P by arrival_time (ascending)
  current_time = 0
  completed_count = 0
  WHILE completed_count < n:
    ready = {p ∈ P : p.arrived ≤ current_time AND p.remaining_time > 0}
    IF ready = ∅:
      CONTINUE
    current_time = current_time + 1
    p* = argmin(ready, key=remaining_time)
    IF p*.start_time = NULL:
      p*.start_time = current_time
    EXECUTE p* for 1 time unit
    p*.remaining_time = p*.remaining_time - 1
    current_time = current_time + 1
    IF p*.remaining_time = 0:
      p*.completion_time = current_time
      COMPUTE TAT, WT, RT
```

Figure 3: Pseudocode for all implemented algorithms

Per-Process Detailed Results Scenario: Presentation

FCFS - Process Results

PID	Arrival (t.u.)	Burst (t.u.)	Start (t.u.)	Completion (t.u.)	Waiting (t.u.)	Turnaround (t.u.)	Response (t.u.)
P1	0	15	0	15	0	15	0
P2	1	3	15	18	14	17	14
P3	2	6	18	24	16	22	16
P4	3	1	24	25	21	22	21
P5	4	8	25	33	21	29	21
P6	5	4	33	37	28	32	28

SJF - Process Results

PID	Arrival (t.u.)	Burst (t.u.)	Start (t.u.)	Completion (t.u.)	Waiting (t.u.)	Turnaround (t.u.)	Response (t.u.)
P1	0	15	0	15	0	15	0
P2	1	3	16	19	15	18	15
P3	2	6	23	29	21	27	21
P4	3	1	15	16	12	13	12
P5	4	8	29	37	25	33	25
P6	5	4	19	23	14	18	14

RR (q=2) - Process Results

PID	Arrival (t.u.)	Burst (t.u.)	Start (t.u.)	Completion (t.u.)	Waiting (t.u.)	Turnaround (t.u.)	Response (t.u.)
P1	0	15	0	37	22	37	0
P2	1	3	2	12	8	11	1
P3	2	6	4	24	16	22	2
P4	3	1	8	9	5	6	5
P5	4	8	9	32	20	28	5
P6	5	4	12	22	13	17	7

Priority - Process Results

PID	Arrival (t.u.)	Burst (t.u.)	Start (t.u.)	Completion (t.u.)	Waiting (t.u.)	Turnaround (t.u.)	Response (t.u.)
P1	0	15	0	15	0	15	0
P2	1	3	15	18	14	17	14
P3	2	6	23	29	21	27	21
P4	3	1	18	19	15	16	15
P5	4	8	29	37	25	33	25
P6	5	4	19	23	14	18	14

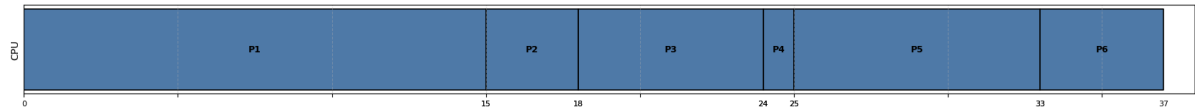
SRTF - Process Results

PID	Arrival (t.u.)	Burst (t.u.)	Start (t.u.)	Completion (t.u.)	Waiting (t.u.)	Turnaround (t.u.)	Response (t.u.)
P1	0	15	0	37	22	37	0
P2	1	3	1	4	0	3	0
P3	2	6	9	15	7	13	7
P4	3	1	4	5	1	2	1
P5	4	8	15	23	11	19	11
P6	5	4	5	9	0	4	0

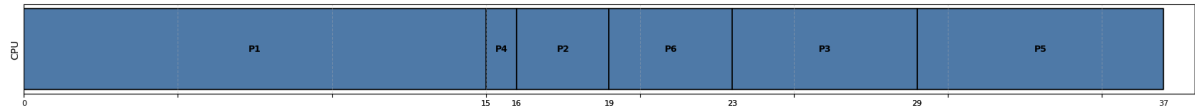
Figure 4: Per-Process Detailed Results

CPU Scheduling — Gantt Charts Scenario: Presentation

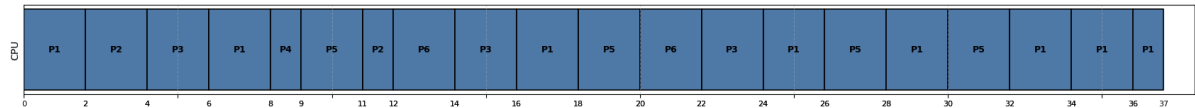
FCFS Gantt Chart



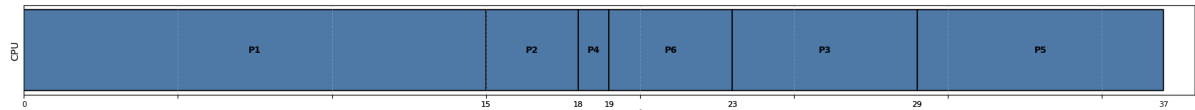
SJF Gantt Chart



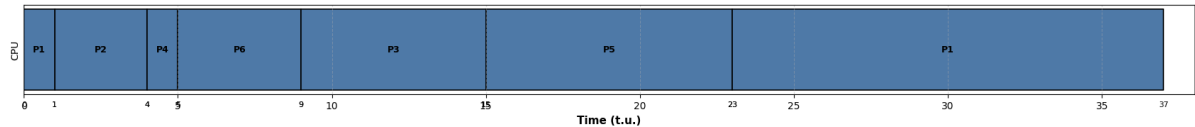
RR (q=2) Gantt Chart



Priority Gantt Chart



SRTF Gantt Chart



Time is measured in abstract time units (t.u.). In real OS, 1 t.u. may correspond to ms or CPU cycles depending on hardware.

Figure 5: Gantt Charts Execution Timelines



Figure 6: Scheduling Metrics Comparison Bar Charts

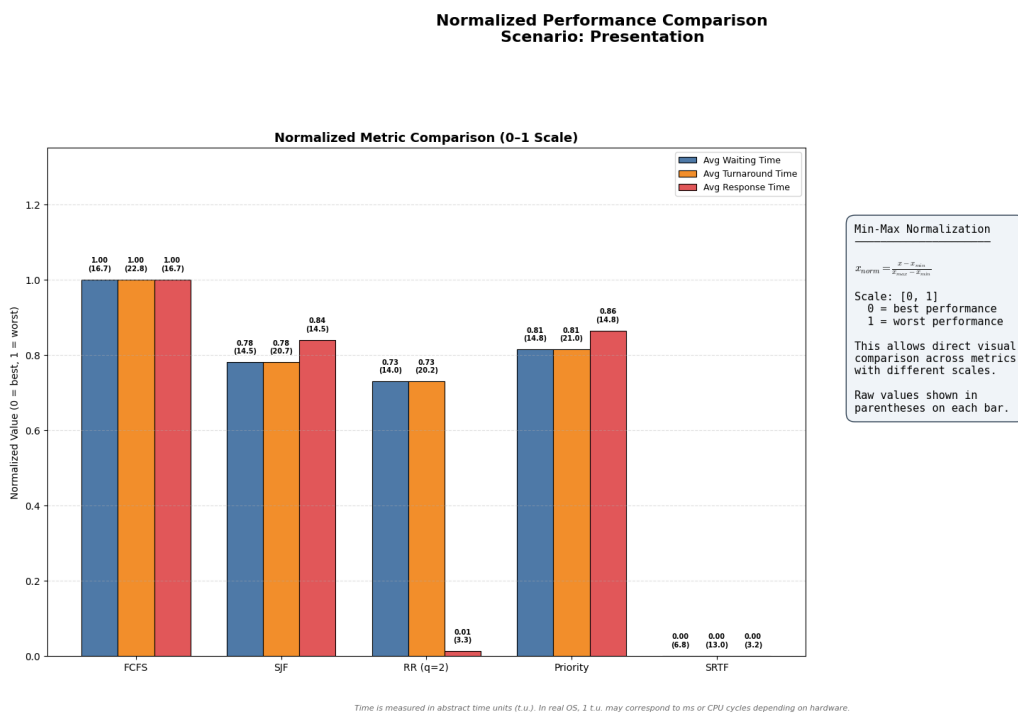


Figure 7: Normalized Performance Comparison

Scheduling Dashboard

Scenario: Presentation

Best Average Waiting Time : SRTF (6.83 t.u.)
 Best Average Turnaround Time : SRTF (13.00 t.u.)
 Best Average Response Time : SRTF (3.17 t.u.)
 Best Throughput : FCFS (0.16 proc/t.u.)

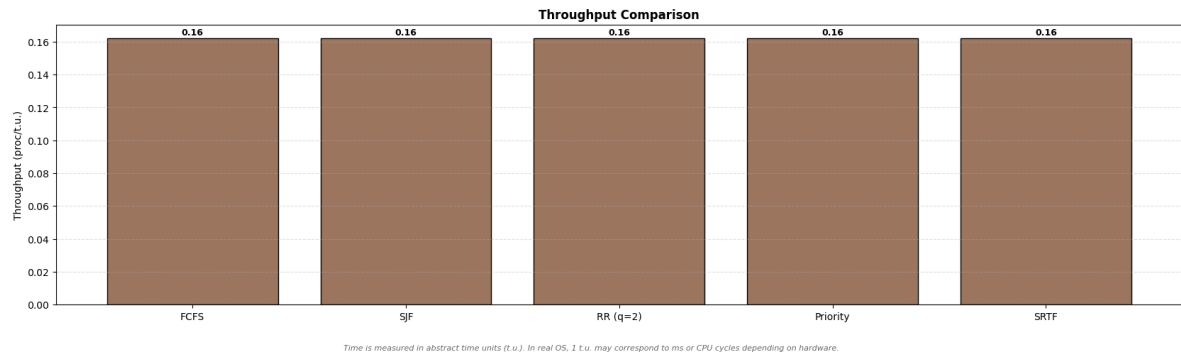


Figure 8: Overall Scheduling Evaluation Dashboard

Result Analysis & Motivation — Presentation

Metric	Best Algorithm	Value	Worst Algorithm	Value
Avg Waiting Time	SRTF	6.83 t.u.	FCFS	16.67 t.u.
Avg Turnaround Time	SRTF	13.00 t.u.	FCFS	22.83 t.u.
Avg Response Time	SRTF	3.17 t.u.	FCFS	16.67 t.u.
Throughput	FCFS	0.1622 proc/t.u.		

Why is SRTF the best for Waiting Time?

SRTF is the preemptive counterpart of SJF and is provably optimal for minimizing average waiting time across all scheduling algorithms (Silberschatz et al., Ch. 5.3.4). When a new shorter process arrives, SRTF immediately preempts the current one, ensuring no process waits unnecessarily behind a longer one.

Why is SRTF the best for Response Time?

SRTF provides excellent response times because short processes arriving mid-execution can preempt longer ones and start immediately.

Throughput Discussion:

Throughput is effectively identical across all algorithms (approx. 0.1622 proc/t.u.). This is expected for a fixed set of n processes; the observation window $\Delta = \max(c_i) - \min(a_i)$ is determined by the workload, not the scheduling order. Throughput differentiation requires continuous process arrivals or multi-queue scenarios.

Figure 9: Result Analysis and Motivation

12 Conclusion

This project's aim was to effectively bridge theoretical process scheduling with practical visualization and implementation, combining programmatic execution with comprehensive statistical analysis. The simulation clearly demonstrates not only how

FCFS, SJF, RR, Priority, and SRTF function, but also some carefully analysis of the results and also showcasing real-time scenarios on user's operating systems.

References

- [1] Silberschatz, Galvin, Gagne — “Operating System Concepts” (10th Ed.), Ch. 5.
- [2] Neetu Goel, Dr. R.B. Garg — “A Comparative Study of CPU Scheduling Algorithms.” *TEERTHANKER MAHAVEER UNIVERSITY*. <https://arxiv.org/abs/1307.4165>
- [3] M. Rajasekhar Reddy et al. — “Comparative Analysis of CPU Scheduling Algorithms and Their Optimal Solutions.” *IEEE Xplore*. <https://ieeexplore.ieee.org/document/8819679>
- [4] “CPU Scheduling Algorithms in Python” — *Scribd OS-LAB Document*. <https://www.scribd.com/document/844965267/OS-LAB>
- [5] Rameen Kashif — “CPU Scheduling Algorithms in Operating Systems.” *Medium Article*. <https://medium.com/@rameenkashif243/cpu-scheduling-algorithms-in-operating-systems-31ede90164b5>
- [6] Process-Scheduler GITHUB REPO — <https://github.com/Emnaghz/Process-Scheduler/>
- [7] os-scheduling-algorithms GITHUB REPO — <https://github.com/MojTabaa4/os-scheduling-algorithms>